

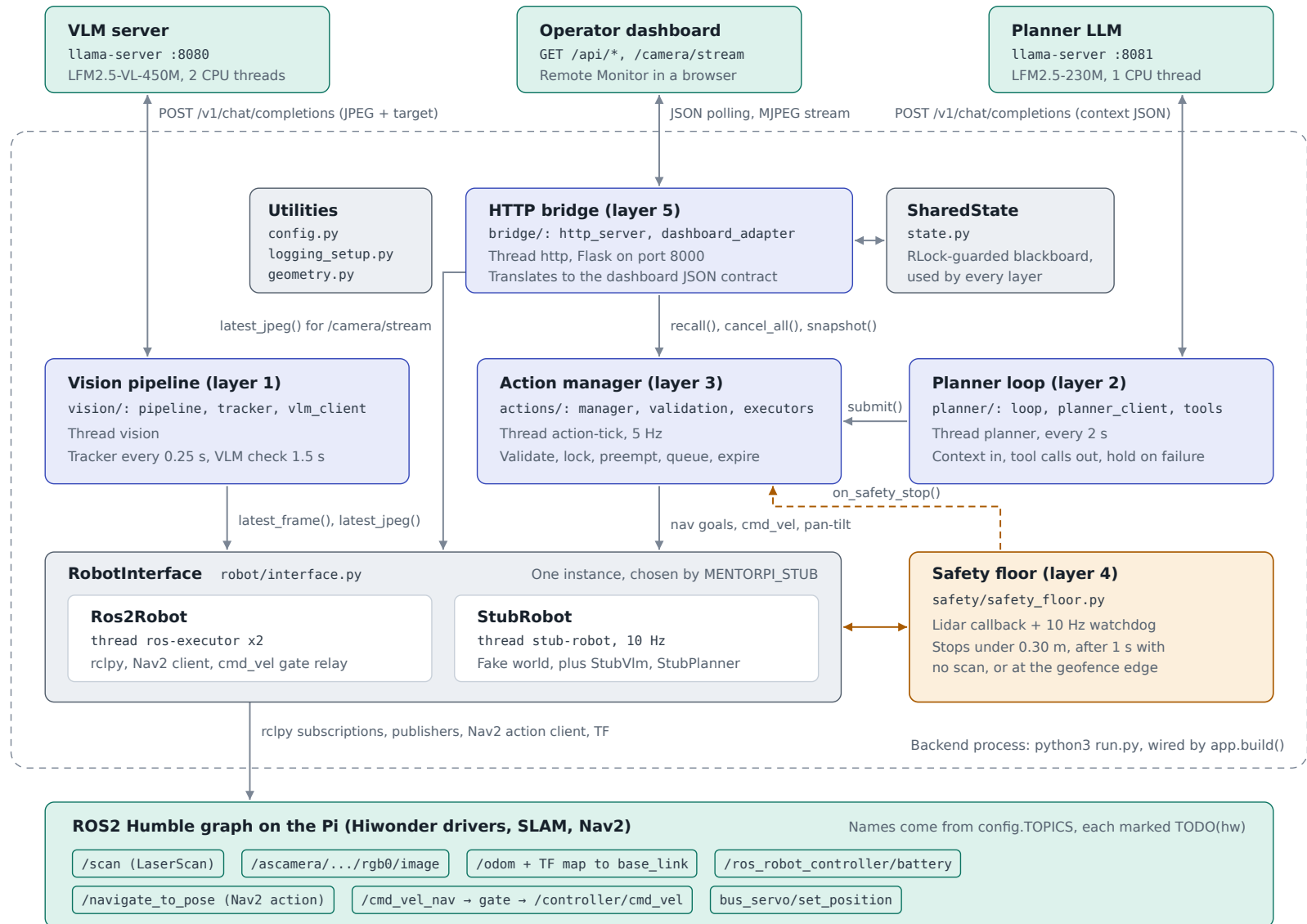
MentorPi backend v1: how the code fits together

A Python process that finds a described target with a Hiwonder MentorPi robot. Two small local LLMs handle seeing and deciding. A deterministic action layer, plus a safety floor the planner cannot touch, decides what the wheels actually do.

Entry point `run.py` calls `mentorpi.app.main()`. That function builds five layers on one `RobotInterface`, starts about eight threads, and serves the dashboard on port 8000. Everything below comes from reading the source in `mentorpi_backend/`.

1. System map
2. Component details
3. Packages and files
4. Threads and timing
5. SharedState hub
6. Goal pipeline and lifecycle
7. A mission, end to end
8. Safety path
9. Interfaces
10. Stub and real modes, tests

System map



■ Separate process or external system
 ■ Backend layer with its own thread
 ■ Safety floor, outside the goal system
 ■ Shared infrastructure

— Safety-originated signal

Component details

The same notes the interactive system map shows when you click a component.

Operator dashboard

mentorpi_monitor HTML (separate file)

Talks to: HTTP bridge on port 8000; Reads: /api/state, /api/goals, /api/vision; Sends: target, behavior, recall

- Polls JSON and shows the MJPEG camera stream.
- Its contract is fixed: dashboard_adapter.py bends backend data to fit it, never the reverse.
- test_dashboard_contract.py guards that contract.

HTTP bridge (layer 5)

bridge/http_server.py,
bridge/dashboard_adapter.py

Thread: http, plus one per request; Classes: create_app(), HttpServer; Holds: state, manager, robot

- POST /api/target cancels live goals, then state.set_target().
- POST /api/recall calls manager.recall(); POST /api/behavior sets follow or conceal.
- GET /camera/stream pulls latest_jpeg() from the robot adapter.
- Debug routes submit or cancel goals directly when MENTORPI_DEBUG=1.

VLM server

scripts/start_models.sh (external llama-server)

Port: 8080, OpenAI-style /v1/chat/completions; Model: LFM2.5-VL-450M Q8_0 + mmproj; Threads: 2 CPU threads, context 2048

- Receives a 320 px JPEG and the target text; must return JSON {match, confidence, bbox}.
- Health checked at /health every 5 s; result lands in health.vision_model.
- In default stub mode StubVlm stands in and fails 5% of calls on purpose.

SharedState

mentorpi/state.py

Lock: threading.RLock; Types: Pose2D, VisionResult; Views: snapshot_state, snapshot_vision, planner_context

- Holds target, recall flag, battery, pose, vision, tracker, last sighting, zones searched, health, safety, behavior and concealment.
- set_vision() also projects a match into map coordinates for last_sighting.
- See the SharedState hub section for every reader and writer.

Planner LLM

scripts/start_models.sh (external llama-server --jinja)

Port: 8081; Model: LFM2.5-230M Q8_0; Threads: 1 CPU thread

- Gets the system prompt from prompts.py and the context JSON as the user message.
- Output constrained by JSON schema (default), GBNF grammar, or native tool calls.
- StubPlanner replaces it in stub mode, failing 2% of calls.

Utilities

config.py, logging_setup.py, geometry.py

config: Env vars, topics, zones, geofence, timings; logging: JSON-lines events via log_event(); geometry: point_in_polygon, wrap_angle, quaternions

- Imported by every layer; they import nothing from the layers.
- Hardware placeholders in config.py are tagged TODO(hw); discover_hw.sh helps fill them.

Vision pipeline (layer 1)

vision/pipeline.py, vision/tracker.py,
vision/vlm_client.py

Thread: vision; Cadence: tracker 0.25 s, VLM 1.5 s;
Depends on: robot, state, Verifier

- Idle unless a target is set and recall is off.
- A VLM match (re)starts the TemplateTracker; two clean 'no' answers drop it.
- Tracker boxes are published as source 'tracker' between VLM checks.
- bearing_from_bbox() turns a box into a bearing for the approach executor.

Safety floor (layer 4)

safety/safety_floor.py

Thread: safety-floor watchdog, 10 Hz; Trigger: every
lidar scan; Can be disabled: No

- Checks, in order: scan timeout 1.0 s, obstacle under 0.30 m (resume 0.45 m), geofence.
- On stop: closes the cmd_vel gate and publishes zeros, every time, for redundancy.
- Then updates SharedState.safety and calls manager.on_safety_stop().
- Not a tool, not a goal, no priority; the planner cannot reach it.

Action manager (layer 3)

actions/manager.py, registry.py,
validation.py, goal.py, executors.py

Thread: action-tick, 5 Hz; Locks: chassis, pan_tilt;
Registry: up to 200 goals

- submit(): register, validate, dedupe, TTL check, arbitrate, then start, queue or reject.
- Higher priority preempts synchronously inside submit(), with no planner round-trip.
- tick(): step executors, expire TTLs, promote the queue by priority then FIFO.
- recall() cancels everything and holds at priority 95; on_safety_stop() aborts chassis goals.

RobotInterface

robot/interface.py

Kind: Abstract base class; Implementations: Ros2Robot,
StubRobot; Data types: LidarSummary, NavFeedback,
NavHandle

- latest_frame(), latest_jpeg(), pose(), lidar(), register_lidar_callback().
- send_nav_goal(), cancel_nav_goal(), publish_cmd_vel(), set_pan_tilt().
- emergency_stop_publish() and set_cmd_vel_gate() exist for the safety floor.
- Everything above this line is hardware-agnostic.

Planner loop (layer 2)

planner/loop.py, planner_client.py,
prompts.py, tools.py, grammar.gbnf

Thread: planner, every 2 s; Timeout: 4 s per decision;
Priority band: 0 to 79

- Skips the cycle with no target or during recall.
- Context is state.planner_context() plus manager.planner_view().
- Each tool call becomes a Goal; sighting-driven goals take the sighting time as evidence.
- On model failure with nothing running, submits hold_position at priority 10.

Ros2Robot

robot/ros2_robot.py

Thread: ros-executor, MultiThreadedExecutor with 2
workers; Mode: MENTORPI_STUB=0; Status: Not yet run
on hardware

- Subscribes to scan, camera, battery and odom with best-effort QoS; polls TF at 2 Hz.
- Relays /cmd_vel_nav to /controller/cmd_vel only while the gate is open.
- Wraps Nav2 NavigateToPose futures into plain feedback and result callbacks.
- Message types are resolved from strings in config.MSG_TYPES.

StubRobot

robot/stub_robot.py

Thread: stub-robot, 10 Hz; Mode: MENTORPI_STUB=1 (default); Extras: StubVIm, StubPlanner

- Simulates motion, lidar and rendered camera frames with a hidden target.
- Lets the whole stack and all 51 tests run on a laptop without ROS2.
- StubVIm reports the target when it is in the simulated view.

ROS2 graph

Hiwonder MentorPi stack on the Raspberry Pi 5

Distro: ROS2 Humble; Nav: Nav2 NavigateToPose; Pose: TF map to base_link, odom fallback

- Topics and message types live in config.TOPICS and config.MSG_TYPES.
- Nav2's controller output should be remapped to /cmd_vel_nav so the gate sits in front of the motors.
- Stock Hiwonder obstacle avoidance stays on as an independent override.

Packages and files

53 files in the archive; 30 Python modules under `mentorpi/`. Each panel lists the main symbols per file and what the package imports, which shows the layering: nothing imports `app.py`, and nothing but `app.py` imports the safety floor.

`run.py, app.py`

Entry point and wiring

`run.py`

Puts the repo on `sys.path` and calls `app.main()`.

`app.py`

`build()` picks stub or real robot and model clients, then constructs `state`, `manager`, `safety`, `vision`, `planner`, `http`. `main()` starts them in that order, waits for `SIGINT` or `SIGTERM`, shuts down in reverse.

Imports every layer.

`bridge/`

Layer 5: HTTP API for the dashboard

`http_server.py`

`create_app(state, manager, robot)` registers Flask routes and CORS. `HttpServer` runs `werkzeug` in its own thread.

`dashboard_adapter.py`

`state_payload`, `goals_payload`, `vision_payload`. Converts seconds to ms, bbox arrays to objects, pending to queue, and preempted aborts to a `PREEMPTED` status.

Imports `actions.manager`, `robot.interface`, `state`, `config`.

`planner/`

Layer 2: decides what to do next

`loop.py`

`PlannerLoop`, `Decider` protocol. `cycle()` builds context, calls `decide()`, submits goals, falls back to `hold_position`.

`planner_client.py`

`PlannerClient`, `parse_planner_output()`, `ToolCall`, `PlannerDecision`. Constrain modes: `json_schema` (default), `gbnf`, `native`.

`prompts.py, tools.py, grammar.gbnf`

Versioned system prompt; tool schemas derived from `actions.goal.ACTIONS`; llama.cpp grammar.

Imports `actions.goal`, `actions.manager`, `actions.validation`, `state`.

vision/

Layer 1: is the target in view, and where

pipeline.py

VisionPipeline, Verifier protocol. One thread, two cadences; two clean "no" answers drop the tracker.

tracker.py

TemplateTracker: OpenCV template match on 160 px grayscale, score ≥ 0.55 , drops after 8 misses.

vlm_client.py

VlmClient.verify() sends a 320 px JPEG;
parse_vlm_output() fails closed on garbage;
bearing_from_bbox().

Imports robot.interface, state.

actions/

Layer 3: turns requests into safe motion

manager.py

ActionManager: submit, tick, recall, cancel_all, on_safety_stop, planner_view.

goal.py, registry.py, validation.py

Goal dataclass, 7 states, about 30 reason codes, the ACTIONS table. GoalRegistry keeps up to 200 goals. validate() runs before any motion.

executors.py

Six executors (one per action) sharing _NavLeg for Nav2 legs.

Imports robot.interface, state, geometry.

safety/

Layer 4: collision stop and geofence

safety_floor.py

SafetyFloor: on_scan() from the lidar callback, _evaluate(), _set_stop(), _watchdog(), read-only is_stopped(). No public disable.

Imports robot.interface, state, geometry.

Imported only by app.py.

robot/

The only code that talks to hardware

interface.py

RobotInterface ABC plus LidarSummary, NavFeedback, NavHandle and nav result constants.

ros2_robot.py

Ros2Robot: rclpy node, message types resolved from strings, Nav2 ActionClient, TF pose polling, cmd_vel gate.

stub_robot.py

StubRobot simulated world, StubVlm (5% failures), StubPlanner (2% failures).

Imports state, geometry; the stub also imports planner types.

state.py, config.py, ...

Shared by all

state.py

SharedState, Pose2D, VisionResult, and snapshot builders for the dashboard and planner.

config.py

Every knob: env vars, topics, zones, geofence, priorities, timings, thresholds. Hardware guesses marked TODO(hw).

logging_setup.py, geometry.py

JSON-lines event logs via log_event(); polygon, angle and quaternion helpers.

Imported everywhere; import nothing from the layers.

tests/, scripts/

Verification and bring-up

tests/

51 pytest tests over the action manager, dashboard contract and output parsers, running against the stub world.

scripts/

start_models.sh launches both llama-servers; discover_hw.sh dumps ROS topics, actions and types to fill in TODO(hw).

Plus README.md, DEPLOY.md, requirements.txt, logs/.

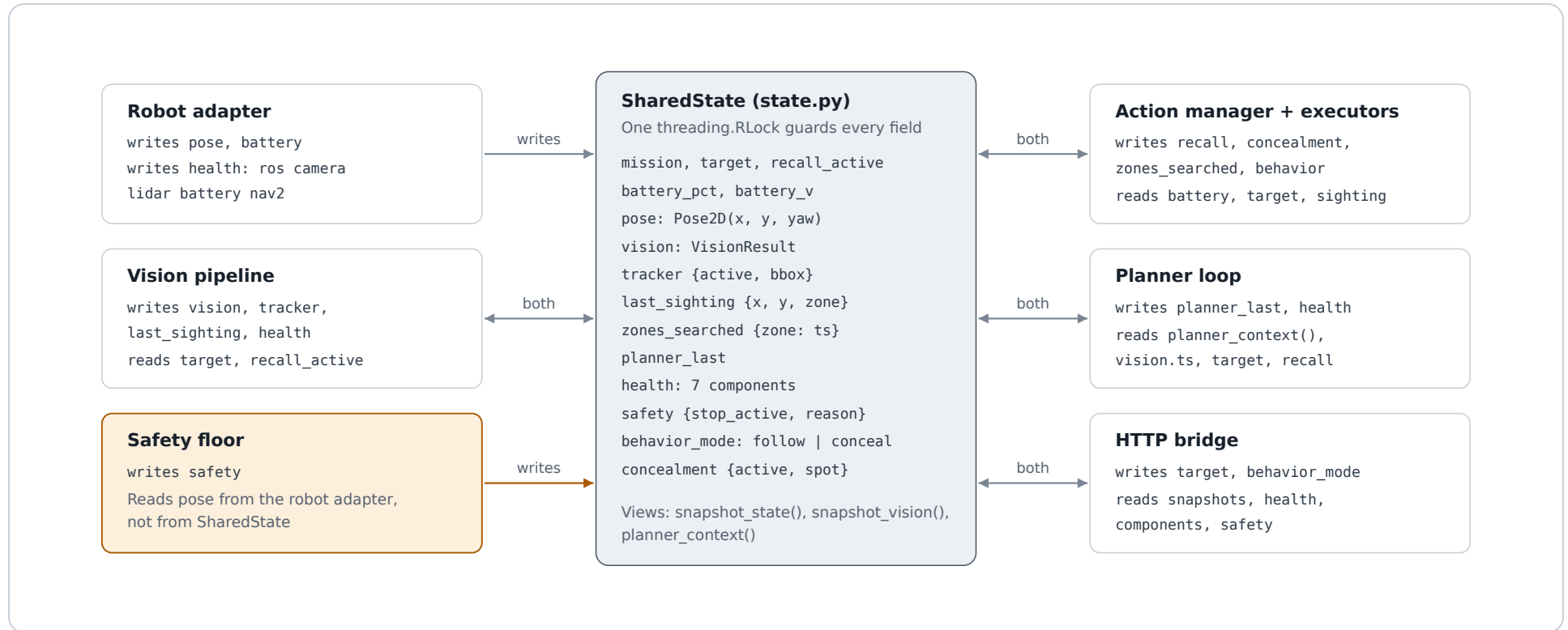
Threads and timing

Each layer owns one thread and communicates through SharedState or direct method calls guarded by locks. No thread other than the ROS executor spins rclpy.

Thread	Owner	Cadence	What it does
ros-executor (2 workers)	Ros2Robot	Event-driven; TF poll 2 Hz; health every 2 s	Runs subscription callbacks (scan, image, odom, battery), Nav2 futures, and the gate relay from /cmd_vel_nav to the motors.
stub-robot	StubRobot	10 Hz	Replaces the executor in stub mode: moves the fake robot, synthesises lidar and camera frames.
safety-floor	SafetyFloor	Every scan, plus 10 Hz watchdog	Evaluates stop conditions on each scan; the watchdog catches missing scans and keeps republishing zero velocity while stopped.
action-tick	ActionManager	5 Hz	Steps executors, enforces TTL on running and queued goals, promotes the queue.
vision	VisionPipeline	Tracker 0.25 s, VLM 1.5 s	Only runs with a target set and no recall. VLM timeout 6 s.
planner	PlannerLoop	2 s	One decision per cycle; timeout 4 s; model health checked every 5 s.
http (+1 per request)	HttpServer	On request	Flask via werkzeug; MJPEG stream holds a request thread per viewer.
main	app.main()	Blocks	Waits on a stop event set by SIGINT or SIGTERM, then shuts down http, planner, vision, manager, safety, robot.

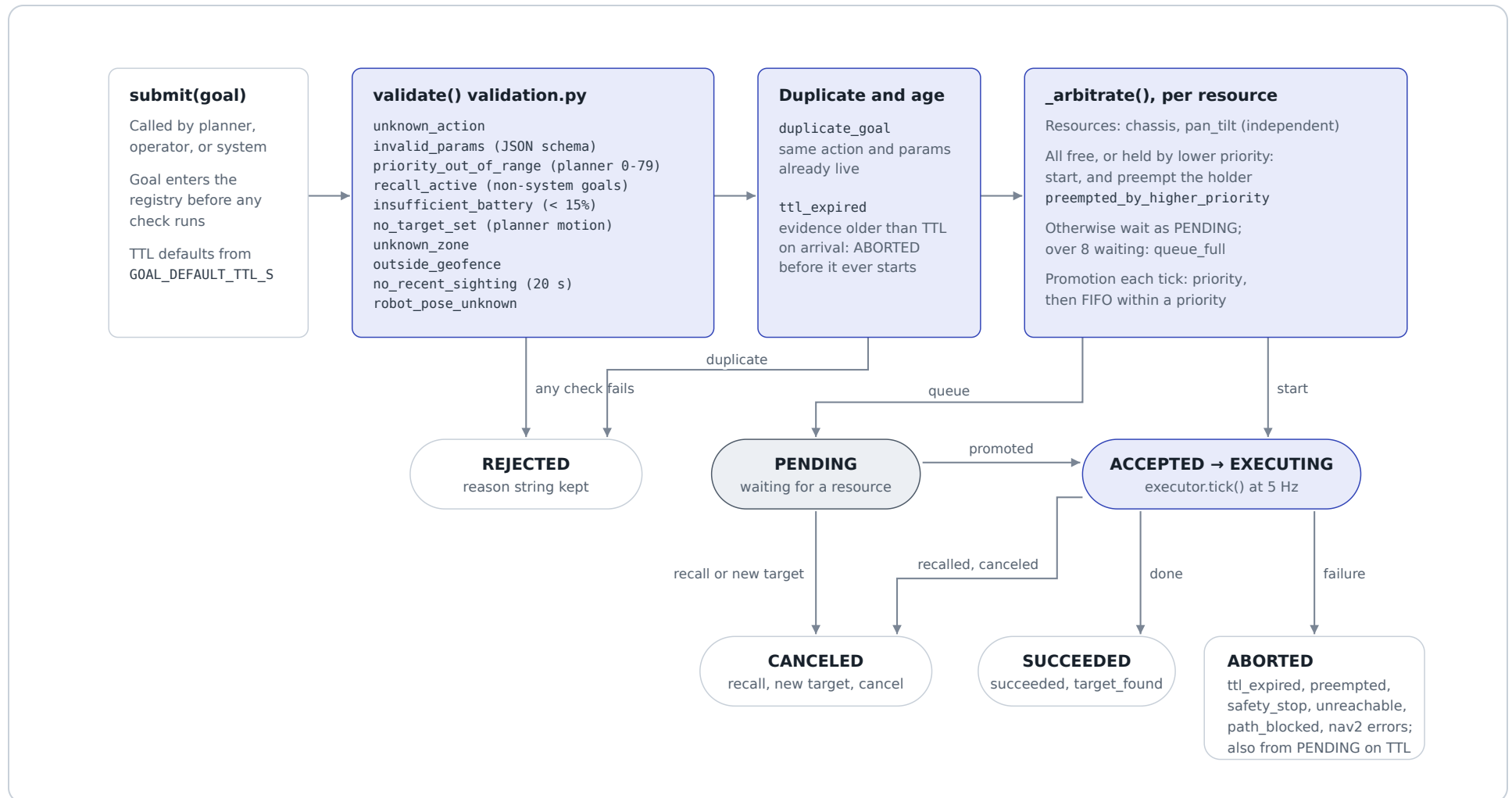
SharedState hub

All shared facts live in one object behind a reentrant lock. Producers on the left mostly write; consumers on the right mostly read. Getters return copies, so readers never hold the lock while they work.



Goal pipeline and lifecycle

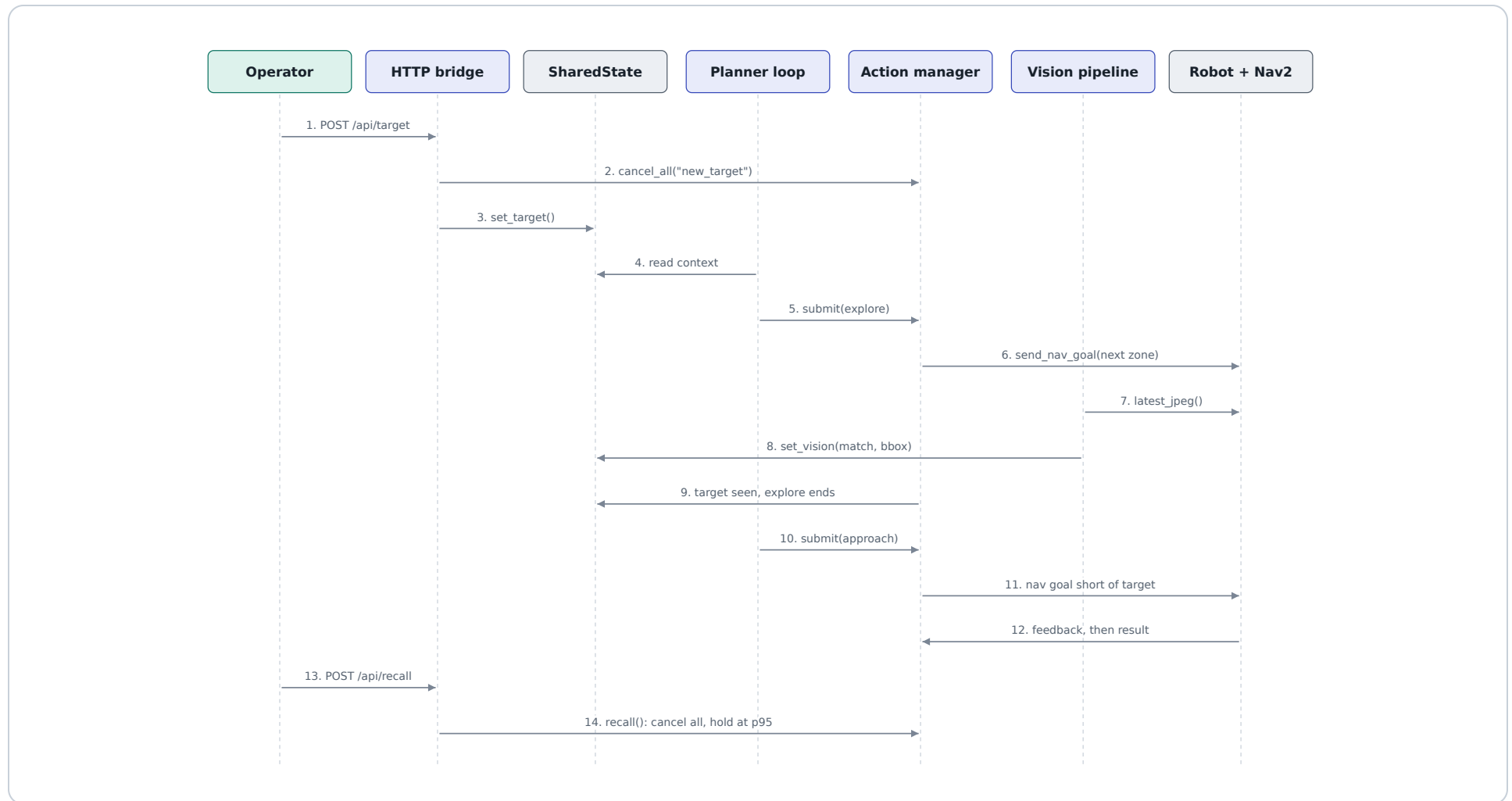
Every motion request, whether from the planner, the operator or the system, becomes a Goal and goes through the same path inside `ActionManager.submit()`. Rejected goals are still recorded with a precise reason, which the planner sees on its next cycle.



Priority bands from `config.py`: planner goals must be 0 to 79; the planner's fallback hold_position uses 10; debug operator goals may use up to 99; the recall hold is a system goal at 95, so nothing the planner sends can displace it. TTL counts from the evidence timestamp, so an approach based on a stale sighting ages out even if it was only just submitted.

A mission, end to end

A typical run in order: the operator names a target, the planner explores, vision finds it, the planner switches to an approach, and the operator recalls.

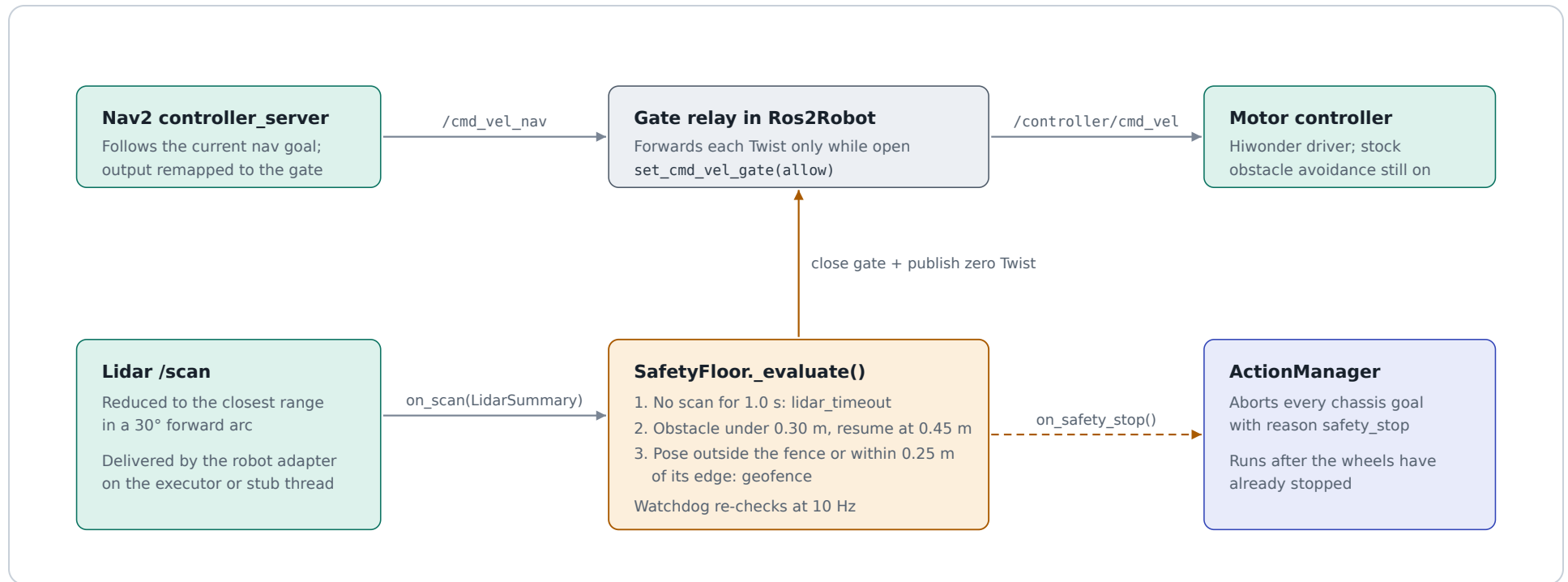


- 1 Operator posts {"target": "red backpack"}. Text is trimmed to 200 characters.
- 2 Any live goals from the previous target are canceled with reason new_target.
- 3 SharedState stores the target and the mission leaves idle. Vision and planner start doing work only from this point.

- 4 Every 2 s the planner reads `planner_context()` plus `manager.planner_view()`, which includes recent goal outcomes and rejection reasons.
- 5 The LLM returns a tool call such as `explore_until_found(ttl_seconds=300)`; it is submitted at planner priority. If the model fails, the loop submits `hold_position` at priority 10 when nothing is running.
- 6 `ExploreUntilFound` sends Nav2 goals to unsearched zones nearest-first and marks each zone searched on arrival.
- 7 Every 1.5 s vision grabs a 320 px JPEG and asks the VLM whether the target is visible. Between checks the template tracker follows the last box every 0.25 s.
- 8 A match writes a `VisionResult`; `SharedState` projects it into map coordinates as `last_sighting`.
- 9 The explore executor sees the sighting and finishes with `target_found`.
- 10 The planner sees the new sighting and asks for `approach_target(standoff_meters=1.0)`. Its TTL counts from the sighting time.
- 11 `ApproachTarget` projects the bearing from the box and the lidar range into the map and sends a goal the standoff distance short, re-planning while the target stays visible.
- 12 Nav2 feedback streams into `goal.feedback` for the dashboard; arrival within the standoff marks the goal SUCCEEDED.
- 13 Operator presses recall. If behavior is set to conceal first, the planner may pick `reposition_for_concealment` instead.
- 14 Recall cancels every goal, sends a zero velocity, blocks non-system goals, and starts a 120 s `hold_position` at priority 95.

Safety path

The planner has wide discretion over strategy and none over safety. The floor is not a tool, not a goal, has no priority and no disable. It stops the wheels first by closing the gate between Nav2 and the motors, then tells the action manager so goals end with an honest reason.



If Nav2's output cannot be remapped to `/cmd_vel_nav`, set `SAFETY_GATE_CMD_VEL=False`. The floor then publishes zeros straight to `/controller/cmd_vel` at 10 Hz and fights Nav2 for the motors. The code calls this a last resort. The stop also sets `SharedState.safety`, which the dashboard shows.

Interfaces

HTTP endpoints (port 8000)

Method	Path	Handler does
GET	/api/state	Mission, target, pose, battery, zones, last sighting, components, safety, behavior, concealment (adapter shape, ms timestamps).
GET	/api/goals	One active goal or null, queue, and history with finished_at.
GET	/api/vision	Latest match, confidence, bbox object, bearing, source (vlm or tracker).
GET	/api/health	Convenience health map, outside the fixed dashboard contract.
GET	/api/debug/state_raw	Internal snapshot before the adapter translates it.
POST	/api/target	Body {"target": "..."} . Cancels goals, sets the target.
POST	/api/behavior	Body {"mode": "follow" "conceal"} . A standing instruction, not a command.
POST	/api/recall	Calls manager.recall() and returns the hold goal.
POST	/api/debug/goal	Submit any goal as operator source, bypassing the planner. Only when MENTORPI_DEBUG=1 (default).
POST	/api/debug/cancel_all	Cancel everything. Debug only.
GET	/api/debug/config	Effective configuration. Debug only.
GET	/camera/stream	MJPEG stream built from robot.latest_jpeg().
GET	/	Index page.

Planner tools (from actions/goal.py ACTIONS)

Action	Parameters	Resource	Moves chassis	Executor behaviour
navigate_to_zone	zone_id: zone_a to zone_d	chassis	Yes	One Nav2 leg to the zone pose; marks it searched.
explore_until_found	ttl_seconds 10-900	chassis	Yes	Unsearched zones nearest-first until a sighting, all zones done, or TTL.
approach_target	standoff_meters 0.5-5.0	chassis	Yes	Needs a sighting under 20 s old and a known pose; re-plans while visible.
reposition_for_concealment	none	chassis	Yes	Nearest configured cover point; success sets concealment active.
hold_position	seconds 1-120 (default 5)	chassis	No	Holds the chassis lock; allowed on low battery.
scan_with_camera	pan_deg, optional tilt_deg	pan_tilt	No	Sets the servo; runs alongside a chassis goal.

ROS2 topics and actions (config.TOPICS)

Key	Default name	Direction	Used for
lidar_scan	/scan	Subscribe	Safety floor and approach range.
camera_image / _compressed	/ascamera/camera_publisher/rgb0/image	Subscribe	Latest frame for vision and MJPEG; decoded with numpy or OpenCV, no cv_bridge.
odom + TF	/odom, map to base	Subscribe / poll 2 Hz	Robot pose; TF preferred, odom fallback.
battery	/ros_robot_controller/battery	Subscribe	Percent and volts for validation and the dashboard.
cmd_vel_nav	/cmd_vel_nav	Subscribe	Nav2 velocity into the gate.
cmd_vel_motor	/controller/cmd_vel	Publish	Gated velocity, zeros on e-stop.

Key	Default name	Direction	Used for
pan_tilt_servo	/ros_robot_controller/bus_servo/set_position	Publish	Camera pan-tilt; if the type is unknown the publisher is skipped.
Nav2 action	/navigate_to_pose (NavigateToPose)	Action client	All chassis motion goals.

Stub and real modes

`app.build()` is the only place the mode matters. Everything above the robot and model clients is identical in both.

Setting	Robot	Vision verifier	Planner decider	Needs
MENTORPI_STUB=1 (default)	StubRobot	StubVlm	StubPlanner	Python, Flask, NumPy, OpenCV
MENTORPI_STUB=1 and MENTORPI_STUB_REAL_MODELS=1	StubRobot	VlmClient	PlannerClient	Both llama-servers running
MENTORPI_STUB=0	Ros2Robot	VlmClient	PlannerClient	ROS2 Humble sourced, Hiwonder stack, Nav2, both servers

Other environment variables: `MENTORPI_HTTP_PORT`, `MENTORPI_VLM_URL`, `MENTORPI_PLANNER_URL`, `MENTORPI_DEBUG`. The `Ros2Robot` module has not been run on hardware yet; its identifiers all come from `config.py` and are marked `TODO(hw)`.

Tests

File	Tests	Covers
<code>test_action_manager.py</code>	26	Every rejection reason, independent resource locks, queueing and promotion order, <code>queue_full</code> , duplicates, preemption, TTL from evidence, nav feedback, explore and hold executors, recall, safety abort, registry eviction, JSON-safe snapshots.

File	Tests	Covers
test_dashboard_contract.py	10	Shapes the dashboard expects: state payload keys, zone status, sighting map coords, single active goal, queue key and PREEMPTED status, finished_at in ms, bbox object, behavior and concealment round trips.
test_parsers.py	15	VLM and planner output parsing: clean JSON, fenced JSON, prose-wrapped JSON, and garbage that must fail closed.
conftest.py		Builds a stub world fixture with state, robot and manager.

Generated from mentorpi_backend_v1 source (files dated September 10 to 22, 2026). Numbers such as distances, rates and priorities are the defaults in config.py and will change as hardware values are filled in.